Project 16: Product Photo Batch Optimizer

Domain: E-commerce / Small Business · **Difficulty:** Intermediate → Advanced

1. One-Line Concept

A batch tool that resizes, converts and compresses product photos in parallel worker threads, compares quality before and after, and downloads everything as a ZIP built by hand.

2. Real-World Problem

Heavy images slow small online stores and hurt conversions, yet many optimisers upload photos to unknown servers or limit batch size. Sellers and web developers use it to prepare fast-loading images privately.

3. Core Features

- Drag-and-drop dozens of images; per-file status list.
- Resize by max width / height or preset (thumbnail, listing, hero).
- Convert to JPEG or WebP with adjustable quality; keep PNG where required.
- Before / after comparison slider with size savings and percentage.
- Parallel processing pool sized to the device's CPU cores.
- Progress, cancel and retry for individual files.
- Filename templates (`product-{n}-{width}w`).
- Download all as a ZIP created in the browser.

4. HTML Skills

- `<input type="file" multiple>` with labelled drop zone
- `<img>` with `srcset` output preview
- `<progress>` per file
- `<input type="range">` for quality
- `role="status"` for batch summary

5. CSS Skills

- Before / after slider using `clip-path` or overlay width
- Grid gallery with status badges
- Progress animations
- Sticky action bar
- Responsive settings panel

6. JavaScript Skills

- Worker pool with a task queue and concurrency limit
- `createImageBitmap` and `OffscreenCanvas` processing
- Canvas `toBlob` with type and quality
- CRC-32 and ZIP 'store' format writer
- Memory management (bitmap `close()`, revoke URLs)
- Cancellation and error recovery

7. Browser APIs / Web APIs

- `Web Workers + OffscreenCanvas` – parallel image processing
- `File API + Drag and Drop API` – input
- `Canvas toBlob` – encode
- `Blob + URL.createObjectURL` – downloads
- `navigator.hardwareConcurrency` – pool size
- `File System Access API` – optional save-to-folder (Chromium)

8. Data / Storage

Browser memory for the batch; LocalStorage for presets and last-used settings. No API, no upload. `data/presets.json` (mock JSON) stores size presets.

9. UI/UX Requirements

Users should understand savings at a glance: total MB saved, per-file percentage and a visual quality check. Never freeze the page during processing. Failed files are reported individually with a clear reason.

10. Responsive Requirements

Device	Expected behaviour
Mobile	Single column; settings in a sheet; list-style results.
Tablet	Settings beside the file list.
Desktop	Settings, gallery and comparison viewer side by side.

11. JavaScript Logic Challenges

1. Build a worker pool that limits concurrency and recovers from a failed task.
2. Encode a valid ZIP archive (headers, CRC-32, central directory) without a library.
3. Keep memory low when processing hundreds of large images.
4. Build an accessible before / after slider that works with keyboard and touch.

12. Advanced Features

1. Automatic quality search: find the smallest size that stays above a similarity threshold.
2. Generate responsive `srcset` sets and ready-to-paste `<picture>` markup.
3. Watermark and background-removal-by-colour options.

13. Difficulty

Intermediate → Advanced

14. Resume Value

Demonstrates Workers, binary formats and memory management on a problem with clear business value. Few beginner portfolios show a hand-built ZIP writer or a worker pool.

Skills Demonstrated: Web Workers & Concurrency · Canvas / OffscreenCanvas · Binary Formats (ZIP) · Performance & Memory · File API & Drag and Drop · Accessible Custom Controls · Batch UX.

15. Suggested Folder Structure

```
photo-batch-optimizer/
├── index.html
├── css/
│   └── style.css
├── js/
│   ├── main.js
│   ├── pool.js
│   ├── worker.js
│   ├── encode.js
│   ├── zip.js
│   ├── crc32.js
│   ├── compare-slider.js
│   └── ui.js
└── data/
    └── presets.json
```